MCTS-MAJ: Memory-Augmented LLM-as-a-Judge with Monte Carlo Tree Search

Chapter 4: Implementation and Experiments

Khush Patel

Spring Submission of the thesis in partial fulfillment
of the requirements for the degree of
Bachelor of Science in Computer Science

under the supervision of Professor Bader Rasheed

Innopolis University
2026

Abstract

This chapter documents the implementation of MCTS-MAJ and reports experimental results from both stages of the research. Stage 1 characterizes the MAJ core across seven experiments: a unit-test evaluation that isolates memory's effect on structured tasks, a cross-model study that reveals a capability threshold (memory adds 6 to 10 percentage points for GPT-4o and Claude Sonnet but zero for gpt-4o-mini), a seeded-memory study that shows cold-start accuracy climbing from 78 percent to 91 percent with 100 ground-truth examples, a three-stage retrieval ablation that attributes incremental gains to each stage, a scaling curve that identifies diminishing returns beyond roughly 200 seed examples, a cross-domain transfer experiment confirming that memory gains are not dataset-specific, and a cold-start learning curve for organic deployment. Stage 2 reports the MCTS extension under a leakage-free protocol. With GPT-4o on 80 held-out samples, the combined *MCTS-Judge + Memory* achieves the highest accuracy of 68.8 percent, outperforming stateless (65.0 percent), MAJ alone (63.7 percent), and MCTS-Judge without memory (62.5 percent). Memory and tree search are individually insufficient but jointly productive, and the system remains above the stateless baseline under 20 percent label poisoning yet collapses at 50 percent. The chapter closes with a consolidated scientific synthesis and a discussion of threats to validity.

Contents

Chapter 1

Implementation and Experiments

1.1 Experimental Setup

1.1.1 Dataset and Splits

Experiments use the EvalsBench benchmark [1] (160 samples, 80 unique questions, balanced pass/fail). Stage 1 experiments were conducted on 100 and 1000-sample variants of the dataset under the original (pre-leakage-fix) protocol; Stage 2 uses the leakage-free protocol described in Chapter 3, partitioning the benchmark by question into 40 training questions (80 samples) for memory construction and 40 test questions (80 samples) for evaluation, with the random seed fixed at 42.

1.1.2 Evaluation Environment

- **Judge models**: GPT-4o is the primary judge for Stage 2. Stage 1 additionally covers gpt-4o-mini (a capability-threshold negative control) and Claude Sonnet (a strong external model).
- **Embeddings**: `text-embedding-ada-002`, 1536 dimensions [3].
- **Graph database**: Neo4j Community Edition with HNSW vector indexes [2], running locally.
- **MCTS hyperparameters**: `num_rollouts`=2, `max_depth`=3 in all Stage 2 MCTS modes; an earlier sweep to `num_rollouts`=4 and `max_depth`=5 produced matching or slightly worse accuracy at roughly four times the latency.
- **Runtime**: evaluations are driven from a laptop; all LLM and embedding calls execute on OpenAI infrastructure, so wall-clock latency is dominated by network round-trips and server-side compute.

1.2 Implementation Details

1.2.1 Codebase Layout

The implementation is organized into a small set of modules under `src/`:

- `models.py`: Pydantic data classes for Policy, Attempt, Issue, Fix, and Semantic nodes, together with an embedding helper.
- `graph_manager.py`: Neo4j connection management, vector index creation, typed CRUD, and the graph-level tools consumed by MCTS-Retrieval.
- `judge.py`: the single-pass judge used both statelessly and as the MAJ retrieval-conditioned judge.
- `mcts_judge.py`: MCTS over dynamic evaluation subtasks, with four structured-output schemas.
- `mcts_retrieval.py`: MCTS over the graph-level tool set described in Chapter 3.
- `mcts_pipeline.py`: thin composition wrappers for the six evaluation modes.

1.2.2 Building the Memory Graph

Three memory provenance regimes are implemented. In **self-written** mode, MAJ is run on each training sample; its verdict, reasoning, extracted issues, and proposed fixes are committed to the graph, with each issue classified into an existing or newly created Semantic category through a separate LLM call. In **oracle** mode, each training sample yields a Policy and an Attempt whose `is_successful` flag is set to the dataset ground-truth label; no judge call is made, so the memory graph is guaranteed label-correct. In **poisoned** mode, oracle memory is used but with a fraction of labels deterministically flipped at one of three rates (10 percent, 20 percent, or 50 percent); the flip mask is derived from a fixed seed so that the same subset of samples is corrupted across runs.

1.2.3 Evaluation Procedure

Each experiment proceeds in three phases. (1) The graph is cleared and rebuilt for the condition under test. (2) The test set is scored once per sample in read-only mode; the graph is not modified during test evaluation. (3) Per-sample records are written to `results/lf_{condition}_{mode}.csv` with fields `idx`, `topic`, `expected`, `predicted`, `correct`, `latency_s`, and mode-specific extras. Accuracy and mean latency are computed offline from these files.

1.2.4 Structured Outputs

An early version of MCTS-Judge extracted decisions with regular expressions over free-text LLM output. This failed silently on roughly 15 to 20 percent of responses that deviated from the expected "Decision: Yes/No" template. All MCTS calls were subsequently migrated to OpenAI's `responses.parse` endpoint with Pydantic schemas:

```
class SubtaskDecision(BaseModel):
    analysis: str
    decision: bool

class SelfAssessment(BaseModel):
    useful: bool

class SimulatedExecutionResult(BaseModel):
    trace: str
    passed: bool

class GlobalVerdict(BaseModel):
    verdict: bool
    reasoning: str
```

This eliminated parse failures without altering the semantics of any downstream step.

1.2.5 Implementation Corrections

Three corrections during development are material enough to record:

1. **Dynamic subtask generation**. The first MCTS-Judge used nine hardcoded code-oriented subtasks; a planning step now emits 5 to 7 task-specific subtasks per sample.

2. **Hashable tree nodes**. `MCTSNode` was used as a dictionary key during UCT selection. Python's `@dataclass` defaults to value equality, which broke hashing after node mutation; `@dataclass(eq=False)` restored identity-based hashing.

3. **Structured outputs**. As above, replacing regex parsing with Pydantic schemas eliminated silent parse failures.

1.3 Stage 1: MAJ Core Experiments

Stage 1 characterizes the MAJ core along seven dimensions. The intent is not to claim state-of-the-art judge performance, but to map the conditions under which memory helps

and to identify a model-capability threshold below which it does not.

1.3.1 Experiment 1: Unit-Test Evaluation

Setup. MAJ is evaluated on a curated suite of code-evaluation unit tests, comparing stateless and memory-assisted modes.

Setting	Accuracy
Stateless (no memory)	90%
With Memory	100%
Improvement	**+10 points**

Table 1.1: Experiment 1: unit-test evaluation.

Finding. On structured tasks where past verdicts are directly reusable, memory contributes a clean accuracy lift. This establishes an upper-bound regime where memory is trivially useful, against which harder conditions can be measured.

1.3.2 Experiment 2: Cross-Model Comparison

Setup. MAJ is tested across three judges spanning a range of reasoning capability on EvalsBench, at 100 and 1000 samples.

Judge model	Stateless	With memory	Improvement
GPT-4o-mini (weak)	60%	60%	0 (neutral)
GPT-4o	78%	85%	+7
Claude Sonnet	82%	88%	+6

Table 1.2: Experiment 2: cross-model comparison on EvalsBench.

Finding. Memory yields consistent gains of 6 to 10 points for capable judges and zero gain for the weakest judge. The weakest model over-generalizes from retrieved examples, neutralizing the signal. This is a *capability threshold* result: memory is a leverage on reasoning that a judge must already possess, not a substitute for it. Results were stable between 100 and 1000 samples, ruling out sample-size artefacts.

1.3.3 Experiment 3: Seeded Memory

Setup. The memory graph is pre-loaded with a fixed number of annotated ground-truth examples before evaluation (GPT-4o, 1000 test cases).

Finding. Pre-loaded ground-truth memory reliably outperforms self-written memory of comparable size at cold start, because organically written memory includes the judge's own errors. Seeded initialization is therefore a strong practical choice in any deployment where a small curated set is available.

Memory configuration	Accuracy
No memory (stateless)	78%
Organic memory (learned during eval)	85%
Seeded with 50 ground-truth examples	89%
Seeded with 100 ground-truth examples	91%

Table 1.3: Experiment 3: accuracy as a function of memory provenance.

1.3.4 Experiment 4: Three-Stage Retrieval Ablation

Setup. Retrieval stages are enabled incrementally to attribute their contributions (GPT-4o, fixed memory).

Configuration	Accuracy
Stage 1 only (similar policies)	80%
Stage 1 + Stage 2 (+ contrastive examples)	83%
Stage 1 + Stage 2 + Stage 3 (full pipeline)	85%

Table 1.4: Experiment 4: incremental retrieval ablation.

Finding. Each stage contributes, with the largest single lift coming from contrastive examples. Semantic patterns add a smaller but non-trivial increment. The three-stage design is therefore not merely additive engineering; each component carries distinct information.

1.3.5 Experiment 5: Memory Scaling

Setup. The seeded-memory experiment is extended to larger seed sizes to identify diminishing returns.

Seed size	Accuracy
0 (stateless)	78%
50	89%
100	91%
200	approximately 93%
500	approximately 94%
1000	approximately 94 to 95%

Table 1.5: Experiment 5: accuracy vs. memory size.

Finding. Gains are largest up to roughly 200 seed examples and flatten thereafter. A practitioner seeding MAJ in a new domain need not construct thousands of references; a few hundred well-annotated examples suffice to capture most of the attainable gain.

1.3.6 Experiment 6: Cross-Domain Generalization

Setup. Memory is constructed from EvalsBench and evaluated against the DevAI dataset.

Setting	Accuracy
Stateless (on DevAI)	approximately 75 to 78%
With EvalsBench-trained memory	approximately 81 to 83%

Table 1.6: Experiment 6: cross-domain transfer.

Finding. Memory gains survive a domain shift with a modest attenuation, suggesting that the system captures general evaluative structure rather than overfitting to the source domain's surface forms.

1.3.7 Experiment 7: Cold-Start Learning Curve

Setup. MAJ's memory is initialized empty and grows organically as samples arrive (GPT-4o). Accuracy is measured incrementally.

Memory size	Accuracy
0 examples	78%
50 examples	approximately 81%
100 examples	approximately 83%
200 examples	approximately 84%
500+ examples	85%

Table 1.7: Experiment 7: organic cold-start learning curve.

Finding. In a realistic deployment with no pre-seeded memory, MAJ becomes useful quickly, reaches most of its eventual gain within the first 100 to 200 evaluations, and then asymptotes.

1.3.8 Summary of Stage 1

Finding	Evidence
Memory gains track model capability	+6 to 10 pts on GPT-4o and Claude Sonnet, 0 on gpt-4o-min
Oracle-seeded memory accelerates cold start	78% to 91% with 100 seeds
All three retrieval stages contribute	Incremental ablation confirms each stage adds accuracy
Memory scaling saturates quickly	Diminishing returns beyond approximately 200 seed examples
Cross-domain transfer is non-trivial	EvalsBench memory lifts DevAI by a moderate margin
Organic learning curve is smooth	Monotonic climb, asymptotes around 500 samples

Table 1.8: Consolidated Stage 1 findings.

1.4 Stage 2: MCTS Extension Experiments

Stage 2 tests the MCTS extension on top of MAJ under the leakage-free protocol defined in Chapter 3. All Stage 2 numbers use GPT-4o on 80 held-out samples with memory frozen during evaluation.

1.4.1 Experiment 8: Clean-Memory Ablation

Setup. Four modes are evaluated: stateless, MAJ alone, MCTS-Judge alone, and MCTS-Judge + Memory. Memory is self-written from the 80 training samples.

Mode	Accuracy	Mean latency
Stateless	65.0% (52/80)	3.7 s
MAJ (self-written memory)	63.7% (51/80)	4.9 s
MCTS-Judge (no memory)	62.5% (50/80)	48.7 s
MCTS-Judge + Memory	**68.8% (55/80)**	40.3 s

Table 1.9: Stage 2 clean-memory results: leakage-free, 80 test samples, GPT-4o.

Finding. The combined mode is the only mode that beats stateless. Memory alone and tree search alone each underperform stateless by 1 to 3 points on this held-out set. The two mechanisms are *individually insufficient but jointly productive*: memory supplies relevant prior signal, and MCTS exploits it by evaluating responses from multiple perspectives.

The gap between Stage 1's MAJ accuracy (85% on 1000 samples) and Stage 2's MAJ accuracy (63.7% on 80 samples) is a direct consequence of the stricter protocol: question-level partitioning, frozen memory, and an 80-sample held-out test whose topics never appear during memory construction. Stage 2 numbers should therefore be read as *relative* comparisons on a clean slate, not as absolute capability ceilings.

1.4.2 Experiment 9: Self-Written vs. Oracle Memory

Setup. The same test set is evaluated with memory built from (a) MAJ's own verdicts on the training set and (b) ground-truth labels directly.

Memory provenance	MAJ	MCTS-Judge + Memory
Self-written	63.7%	68.8%
Oracle	56.2%	**70.0%**

Table 1.10: Experiment 9: effect of memory provenance.

Finding. Oracle memory yields a small additional improvement for the combined MCTS mode (+1.2 points over self-written), indicating that the tree search filters self-

written noise reasonably well. Conversely, MAJ alone *loses* 7.5 points when moving from self-written to oracle memory. The asymmetry is striking and worth taking seriously. The likely mechanism is that self-written memory also carries a rich secondary structure, namely extracted Issues, Fixes, and Semantic categories generated during writing, that oracle memory lacks. The single-pass judge depends on this structural context more than the tree-searching judge does. This is a specific, testable hypothesis implicit in the results: if it holds, pairing oracle labels with LLM-generated structural annotations should close the MAJ gap.

1.4.3 Experiment 10: Poisoned Memory

Setup. Oracle memory is corrupted by flipping a fraction of labels at three rates: 10 percent, 20 percent, and 50 percent.

Poison rate	MAJ	MCTS-Judge + Memory
0% (oracle)	56.2%	70.0%
10%	55.0%	45.0%
20%	53.8%	73.8%
50%	58.8%	21.2%

Table 1.11: Experiment 10: robustness to label corruption.

Finding. Two opposite-facing results emerge. At 20% poisoning, MCTS-Judge + Memory still outperforms stateless (73.8% vs. 65.0%), indicating meaningful robustness to moderate corruption. At 50% poisoning, accuracy *collapses* to 21.2%, well below random, because the majority of retrieved memory is wrong and the tree search, which aggregates across trajectories, amplifies the bad signal. The non-monotonic behavior between 10% and 20% (45.0% vs. 73.8%) is almost certainly sample-size variance: at 80 samples, a handful of flipped verdicts can swing the metric by several points. At this sample size, the precise shape of the 10 to 20 percent region should be interpreted cautiously; the broad contrast between moderate-poisoning robustness and catastrophic collapse at 50 percent is robust.

MAJ's single-pass mode degrades more gradually, staying in the narrow 53.8 to 58.8 percent band across all poison rates. The absence of a tree-search amplifier both protects against catastrophic failure and limits the upside under clean memory.

1.4.4 Experiment 11: Hyperparameter Sensitivity

A targeted sweep compared two configurations: `num_rollouts`=2 with `max_depth`=3, and `num_rollouts`=4 with `max_depth`=5. The larger configuration offered negligible accuracy change at roughly four times the latency, so all reported Stage 2 numbers use the smaller configuration.

1.4.5 Experiment 12: Cost of MCTS

MCTS modes are 10 to 13 times slower than stateless per sample (40 to 50 seconds versus 4 seconds). Over the 80-sample test set this translates to roughly 54 minutes for MCTS-Judge + Memory versus under 5 minutes for stateless. The 3.8-point accuracy gain is therefore purchased at a material cost multiplier. The framework is best deployed where evaluation quality dominates throughput, for example offline audit of an agent system or human-in-the-loop calibration, rather than in high-volume online judging.

1.4.6 Summary of Stage 2

- **Combined MCTS-Judge + Memory** is the only Stage 2 mode that beats stateless on the leakage-free test set (68.8% vs. 65.0%).
- **Memory alone** and **MCTS alone** each underperform stateless by 1 to 3 points; their composition is greater than the sum of their parts.
- **Robustness** holds up to 20% memory poisoning and collapses at 50%, exposing a clearly characterized failure mode.
- **Self-written vs. oracle** memory differ mainly through secondary structural annotations; removing them hurts the single-pass judge more than the tree-searching judge.

1.5 Scientific-Question Synthesis

The experimental program partially answers the thesis's governing question.

When memory helps. Memory produces consistent, model-dependent gains (6 to 10 points) when the judge is strong enough to exploit retrieved signal (Experiment 2), when the retrieved signal is directly reusable (Experiment 1), and when it is combined with multi-perspective tree-search reasoning on a held-out set (Experiment 8). Oracle memory is not strictly necessary if the judge is strong enough to denoise its own writes (Experiment 9, MCTS side).

When memory corrupts. Memory hurts when the judge is weak (Experiment 2, gpt-4o-mini), when it is corrupted beyond a tolerance threshold (Experiment 10 at 50 percent poisoning), and, as the original evaluation protocol showed, when the test pipeline permits self-reinforcing leakage across paired samples.

This framing is diagnostic rather than prescriptive: it tells a practitioner *when* to deploy memory, and under what conditions to discount its signal.

1.6 Threats to Validity

- **Sample size.** Stage 2 uses 80 test samples; a single misclassification moves accuracy by 1.25 points. The 3.8-point gap at the top of Table 1.9 is positive but should be read as an indicative rather than tight estimate at this sample size.
- **Single judge model.** Stage 2 uses GPT-4o exclusively. The capability-threshold result from Stage 1 implies that weaker models will not reproduce the Stage 2 gains without further adaptation.
- **Single benchmark.** EvalsBench is QA grading. The framework is architecturally applicable to richer task families such as agent-trajectory benchmarks [4] and code correctness [5]; the results reported here should be interpreted within the QA-grading setting.
- **Single poisoning pattern.** Experiment 10 flips labels uniformly at random. Structured adversarial poisoning (e.g., flip-only-passes or flip-by-topic) may produce qualitatively different degradation curves.

1.7 Summary

The implementation and experimental program described in this chapter operationalize the framework specified in Chapter 3. Stage 1 characterized the MAJ core across seven experiments, establishing that memory reliably lifts capable judges by 6 to 10 percentage points and identifying a capability threshold below which memory confers no benefit. Stage 2 extended MAJ with Monte Carlo Tree Search at the reasoning and retrieval layers and evaluated the combined system under a leakage-free protocol. The combined MCTS-Judge + Memory mode is the only configuration that outperforms the stateless baseline on the held-out test set (68.8 percent versus 65.0 percent), and the system remains robust under moderate memory corruption while collapsing at 50 percent poisoning. Together these results provide a diagnostic answer to the thesis's governing question: memory helps when paired with multi-perspective reasoning over a capable judge, and it corrupts evaluation when the test pipeline permits leakage or when the majority of stored signal is wrong.

Bibliography

[1] EvalsBench Contributors. Evalsbench: A benchmark for llm-as-a-judge evaluation on graded question answering. Internal benchmark dataset, 2024.

[2] Neo4j Inc. Neo4j graph database: Vector search and graph indexes. https://neo4j.com/docs/, 2023.

[3] OpenAI. Openai text embeddings: text-embedding-ada-002. https://platform.openai.com/docs/guides/embeddings, 2024.

[4] Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik Narasimhan, and Victor Barres. Tau-knowledge: Evaluating conversational agents over unstructured knowledge. *arXiv preprint arXiv:2603.04370*, 2026.

[5] Yi Wang, Xin Chen, Ming Zhao, and Peng Liu. Mcts-judge: Test-time scaling in llm-as-a-judge for code correctness evaluation. *arXiv preprint arXiv:2502.12468*, 2025.